

Consistency and Replication

Distributed Systems

Major issues:

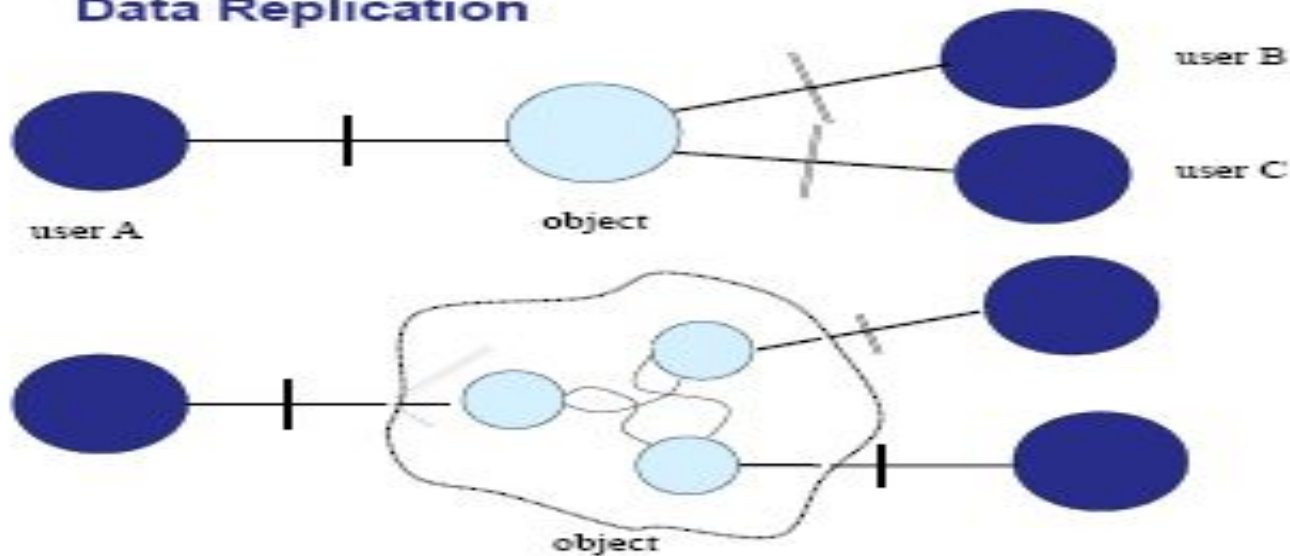
- **Transparency:** Hide the complexity of multiple computers from the user.

Why transparency is challenging because the system must hide: network delays, failures, replication conflicts, consistency issues

- **Scalability:** System should handle growth without slowing down

Why scalability is difficult because when system grows: network traffic increases, synchronization becomes hard, consistency becomes complex, replication overhead increases

Data Replication



Data replication stores multiple copies of an object at different locations to improve performance and availability, but it introduces the challenge of maintaining consistency among replicas.

Advantages

- Faster access (low latency)
- Better availability
- Load is shared
- System becomes scalable

Why replication?

- To enhance reliability
 - If one replica is unavailable or crashes, use another
 - Protect against corrupted data
- To improve performance
 - Scale with size of the distributed system (replicated web servers)
 - Scale in geographically distributed systems

Problems in replication?

Having multiple copies may lead to consistency problems.

- To keep the replicas consistent: meaning if one copy is updated, we need to ensure that the other copies are updated as well, otherwise, the replica will no longer be the same.

To keep replicas consistent, we generally need to ensure that all conflicting operations are done in the same order everywhere

Conflicting operations: From the world of transactions:

Read–write conflict: a read operation and a write operation act concurrently

Write–write conflict: two concurrent write operations

Consistency and Replication

Exactly when and how the modifications need to be carried out determines the price of replication.

Price paid for replication- consistency problems, cached pages outdated, invalidate prev. pages

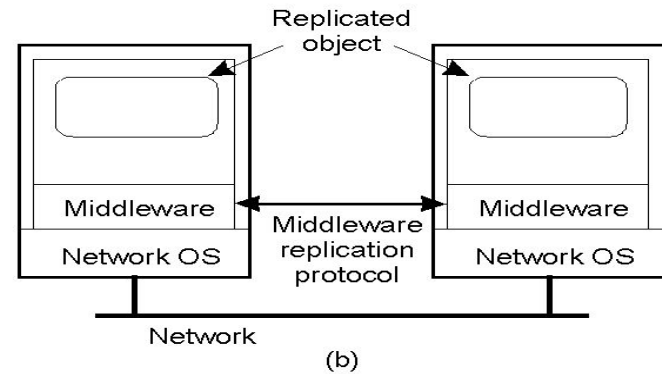
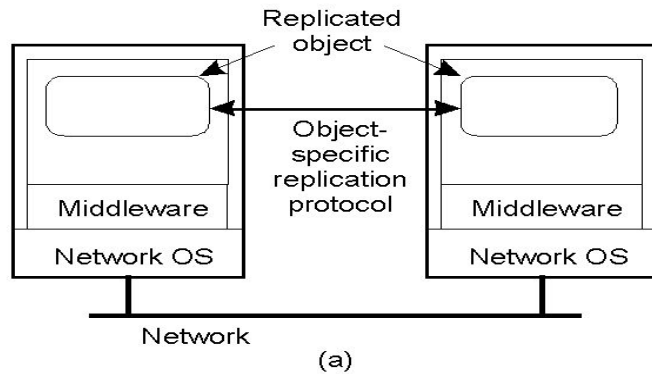
Guaranteeing global ordering on conflicting operations may be a costly operation, downgrading scalability

Replication Issues

- When to replicate?
- How many replicas to create?
- Where should the replicas located?
- How to maintain *consistency*?

Key issue: need to maintain *consistency* of replicated data

Object Replication



- Approach 1: application is responsible for replication
 - Application needs to handle consistency issues.
 - Example Suppose you build a banking app. your banking code must ensure that all servers update balance, conflicts are resolved, order is maintained. So developer does extra work.
- Approach 2: system software (middleware) handles replication
 - Consistency issues are handled by the middleware
 - Simplifies application development but makes object-specific solutions harder
 - In banking app, Your code just updates balance. Middleware automatically sends updates to other servers, maintains consistency, resolves conflicts, Developer is relaxed.

Replication and Scaling

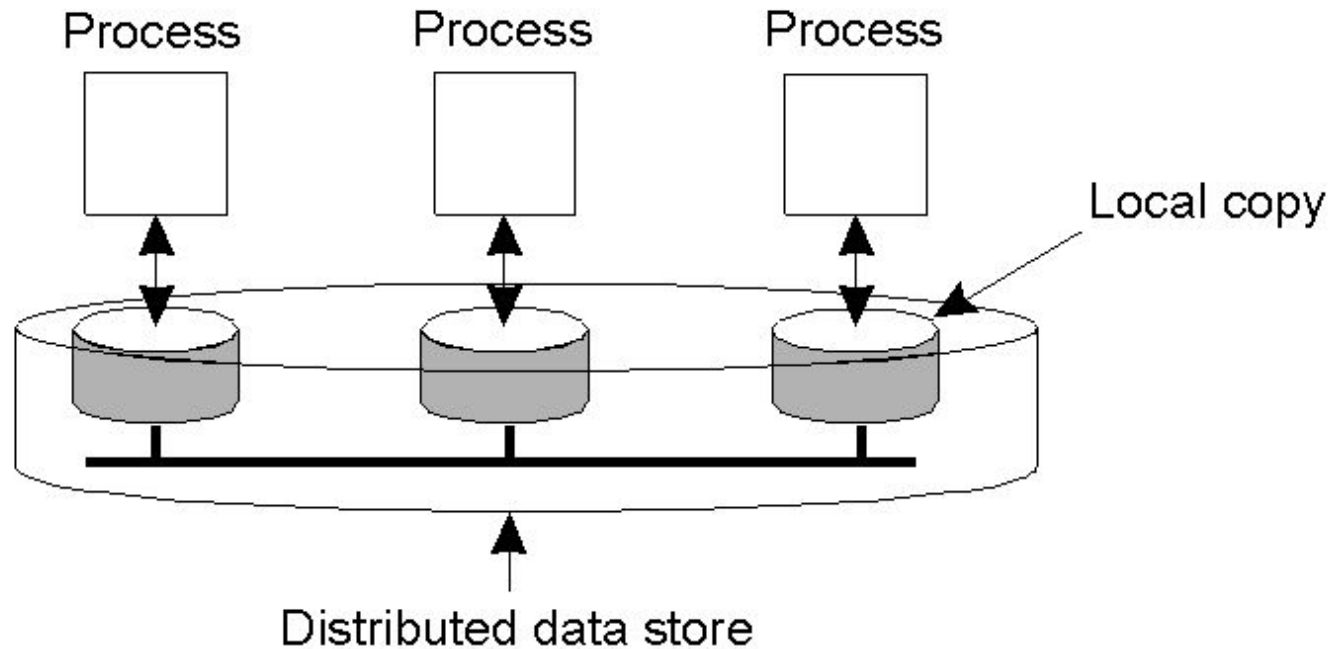
- Replication and caching used for system scalability
- Multiple copies:
 - Improves performance by reducing access times
 - But higher network overheads of maintaining consistency
 - Example: object is replicated N times
 - Read frequency R , write frequency W
 - If $R \ll W$, high consistency overhead and wasted messages
 - Consistency maintenance is itself an issue
 - What semantics to provide?
 - Tight consistency requires globally synchronized clocks!
- Solution: loosen consistency requirements
 - Variety of consistency semantics possible

Consistency models

– Consistency models

- **Data-centric consistency models:** Focus on the whole system's data. They define rules about how all replicas must behave globally. Concerned with overall data correctness. All users should see data according to system rules. Used when strong global consistency is important
- **Client-centric consistency models:** Focus on what one user (client) experiences. Instead of strict global rules, it ensures a good experience for each client. Concerned with individual user experience. Allows temporary differences between replicas. More practical for large distributed systems
 - **Consistency model:** Rules that define when updates become visible.
 - **Data-centric consistency:** Ensures system-wide data correctness.
 - **Client-centric consistency:** Ensures a single client sees reasonable and expected data.

Data-Centric Consistency Models



- Consistency model (aka *consistency semantics*)
 - Contract between processes and the data store
 - If processes obey certain rules, data store will work correctly
 - All models attempt to return the results of the last write for a read operation
 - Differ in how “last” write is determined/defined

Strict Consistency

Any read always returns the result of the most recent write

- Implicitly assumes the presence of a global clock
- A write is immediately visible to all processes
 - Difficult to achieve in real systems (network delays can be variable)

Strict Consistency

Strict consistency means:

Whenever someone writes new data, everyone immediately sees the latest value.
There is no delay at all.

P1:	W(x)a	
P2:		R(x)a

(a)

P1:	W(x)a	
P2:		R(x)NIL R(x)a

(b)

Behavior of two processes, operating on the same data item x.

- A strictly consistent store.
- A store that is not strictly consistent.

Thus in a strictly consistent data store, all writes are instantaneously visible to all processes and an absolute global time is maintained.

Sequential Consistency

- Strict consistency- impossible to implement
- Programmer can manage well with weaker models
- When events in the same process occur very quickly, such that the process is not able to do something else, then the order of statement does not matter.
- When order is essential-semaphores and other synch. Methods used.
- Hence learn to live with weaker consistency.

Sequential Consistency

- The result of any execution is the same as if the (read and write) operations by all processors on the data were executed in some sequential order and the operations of each individual process appear in this sequence in the order specified by its program.
- When processes run concurrently on different m/cs, any valid **interleaving of read and write operations is acceptable**, but *all processes see the same interleaving of operations*

Sequential Consistency

- Sequential consistency: weaker than strict consistency
 - Assumes all operations are executed in some sequential order and each process issues operations in program order
 - Any valid interleaving is allowed
 - All agree on the same interleaving
 - Each process preserves its program order
 - Nothing is said about “most recent write”

Sequential Consistency

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

- a) A sequentially consistent data store.
- b) A data store that is not sequentially consistent.

- Is the following data store sequentially consistent?

A	W(x)a	W(x)b
B	R(x)a	R(x)b
C		R(x)b R(x)a

Linearizability

- Weaker than strict but stronger than sequential
- Operations are assumed to receive a timestamp using a globally available clock, but with finite precision. (i.e. with the assumption that processes use loosely synch. Clocks)
- Assumes sequential consistency *and*
 - If $TS(x) < TS(y)$ then $OP(x)$ should precede $OP(y)$ in the sequence

Linearizability

Stronger than sequential consistency

Difference between linearizability and serializability?

- Granularity: reads/writes versus transactions

Example: assignment : write operation (atomic)

print : read operation (atomic)

Process P1	Process P2	Process P3
x = 1; print (y, z);	y = 1; print (x, z);	z = 1; print (x, y);

Linearizability and Sequential Consistency

Example

- Four valid execution sequences for the processes of the previous slide. The vertical axis is time.

```
x = 1;  
print (y, z);  
y = 1;  
print (x, z);  
z = 1;  
print (x, y);
```

Prints: 001011

Signature:
001011
(a)

```
x = 1;  
y = 1;  
print (x,z);  
print(y, z);  
z = 1;  
print (x, y);
```

Prints: 101011

Signature:
101011
(b)

```
y = 1;  
z = 1;  
print (x, y);  
print (x, z);  
x = 1;  
print (y, z);
```

Prints: 010111

Signature:
110101
(c)

```
y = 1;  
x = 1;  
z = 1;  
print (x, z);  
print (y, z);  
print (x, y);
```

Prints: 111111

Signature:
111111
(d)

- Is 000000 possible ?
- Is 001001 permitted?

- Is 000000 possible ?
- Is 001001 permitted?

- Is 001001 permitted?
- A more subtle example is 001001. The first two bits, 00, mean that y and z were both 0 when P1 did its printing. This situation occurs only when P1 executes both statements before P2 or P3 starts.
- The next two bits, 10, mean that P2 must run after P1 has started but before P3 has started.
- The last two bits, 01, mean that P3 must complete before P1 starts, but we have already seen that P1 must go first. Therefore, 001001 is not allowed.

- Consider the following programs executes by P1 and P2
- P1: {initially $x=0$ } $x := 1$; print y
- P2: {initially $y=0$ } $y := 1$; print x

If sequential consistency is maintained, then what value(s) of (x, y) will **never be** printed?

Linearizability and Sequential Consistency

- Various valid statement orderings may produce different program results that are allowed under the assumption of sequential consistency.
- **The contract between processes and distributed shared data store is that the processes must accept all of these as valid results.**

Causal Consistency

Necessary condition:

- Writes that are potentially casually related must be seen by all processes in the same order.
- Concurrent writes may be seen in a different order on different machines.

Causal Consistency

P1:	W(x)a			W(x)c
P2:		R(x)a	W(x)b	
P3:		R(x)a		R(x)c
P4:		R(x)a		R(x)b

This sequence is allowed with a casually-consistent store, but not with sequentially or strictly consistent store.

Causal Consistency

P1:	W(x)a		
P2:		R(x)a	W(x)b
P3:			R(x)b
P4:			R(x)a

(a)

P1:	W(x)a		
P2:			W(x)b
P3:			R(x)b
P4:			R(x)a

(b)

- a) A violation of a casually-consistent store.
- b) A correct sequence of events in a casually-consistent store.

FIFO Consistency

- Necessary Condition:
Writes done by a single process are seen by all other processes in the order in which they were issued, but writes from different processes may be seen in a different order by different processes.
- Relaxes causal consistency
- Simple implementation: tag each write by (Proc ID, seq #)

FIFO Consistency

If one process writes multiple values,
everyone must see those writes in the same order. BUT...
Writes from different processes may appear in different
orders.

P1:	W(x)a			
P2:	R(x)a	W(x)b	W(x)c	
P3:			R(x)b	R(x)a R(x)c
P4:			R(x)a	R(x)b R(x)c

A valid sequence of events of FIFO consistency

FIFO Consistency

```
x = 1;  
print (y, z);  
y = 1;  
print(x, z);  
z = 1;  
print (x, y);
```

Prints: 00

(a) P1's view

```
x = 1;  
y = 1;  
print(x, z);  
print ( y, z);  
z = 1;  
print (x, y);
```

Prints: 10

(b) P2's view

```
y = 1;  
print (x, z);  
z = 1;  
print (x, y);  
x = 1;  
print (y, z);
```

Prints: 01

(c) P3's view

Statement execution as seen by the three processes from the previous slide.
The statements in bold are the ones that generate the output shown.

FIFO Consistency

Process P1

```
x = 1;  
if (y == 0) kill (P2);
```

Process P2

```
y = 1;  
if (x == 0) kill (P1);
```

Two concurrent processes.

- The key difference between sequential consistency and FIFO consistency is that with the former , although the order of statement execution is non-deterministic, atleast all processes agree what it is. With the latter, they need not agree

Other models

- Even FIFO consistency may be too strong!
 - Requires all writes from a process be seen in order
- Assume use of critical sections for updates
 - Send final result of critical section everywhere
 - Do not worry about propagating intermediate results
 - Assume presence of synchronization primitives to define semantics

Weak Consistency

Properties:

- Accesses to synchronization variables associated with a data store are sequentially consistent
- No operation on a synchronization variable is allowed to be performed until all previous writes have been completed everywhere
- No read or write operation on data items are allowed to be performed until all previous operations to synchronization variables have been performed.

Weak Consistency

- Unlike previous models, weak consistency enforces consistency on a group of operations, not on individual reads and writes
- It only limits the time when consistency holds, rather than the form of consistency

Weak Consistency

```
int a, b, c, d, e, x, y;           /* variables */
int *p, *q;                         /* pointers */
int f( int *p, int *q);             /* function prototype */

a = x * x;                          /* a stored in register */
b = y * y;                          /* b as well */
c = a*a*a + b*b + a * b;           /* used later */
d = a * a * c;                     /* used later */
p = &a;                             /* p gets address of a */
q = &b;                             /* q gets address of b */
e = f(p, q)                         /* function call */
```

A program fragment in which some variables may be kept in registers.

Weak Consistency

P1:	W(x)a	W(x)b	S						
P2:				R(x)a	R(x)b	S			
P3:				R(x)b	R(x)a	S			

(a)

P1:	W(x)a	W(x)b	S						
P2:					S	R(x)a			

(b)

- a) A valid sequence of events for weak consistency.
- b) An invalid sequence for weak consistency.

Other Models

- Entry and release consistency
 - Assume shared data are made consistent at entry or exit points of critical sections

Release Consistency

P1: Acq(L) W(x)a W(x)b Rel(L)

P2: $Acq(L) \quad R(x)b \quad Rel(L)$

P3: $R(x)a$

A valid event sequence for release consistency.

Release Consistency

Rules:

- Before a read or write operation on shared data is performed, all previous acquires done by the process must have completed successfully.
- Before a release is allowed to be performed, all previous reads and writes by the process must have completed
- Accesses to synchronization variables are FIFO consistent (sequential consistency is not required).

Entry Consistency

P1:	Acq(Lx)	W(x)a	Acq(Ly)	W(y)b	Rel(Lx)	Rel(Ly)	
P2:					Acq(Lx)	R(x)a	R(y)NIL
P3:						Acq(Ly)	R(y)b

A valid event sequence for entry consistency.

Entry Consistency

Conditions:

- An acquire access of a synchronization variable is not allowed to perform with respect to a process until all updates to the guarded shared data have been performed with respect to that process.
- Before an exclusive mode access to a synchronization variable by a process is allowed to perform with respect to that process, no other process may hold the synchronization variable, not even in nonexclusive mode.
- After an exclusive mode access to a synchronization variable has been performed, any other process's next nonexclusive mode access to that synchronization variable may not be performed until it has performed with respect to that variable's owner.

Summary of Data-centric Consistency Models

Consistency	Description
Strict	Absolute time ordering of all shared accesses matters.
Linearizability	All processes must see all shared accesses in the same order. Accesses are furthermore ordered according to a (non unique) global timestamp
Sequential	All processes see all shared accesses in the same order. Accesses are not ordered in time
Causal	All processes see causally-related shared accesses in the same order.
FIFO	All processes see writes from each other in the order they were used. Writes from different processes may not always be seen in that order

(a)

Consistency	Description
Weak	Shared data can be counted on to be consistent only after a synchronization is done
Release	Shared data are made consistent when a critical region is exited
Entry	Shared data pertaining to a critical region are made consistent when a critical region is entered.

(b)